

PHASE 41-43: Schema + Monitoring + API Docs

Data: 2026-04-22

Status:  Completo e Deployado

PHASE 41 - Correção de Schema

Mudanças no Prisma Schema

AuditLog Model

```
model AuditLog {  
  // ... campos existentes  
  restaurantId String? // NOVO - opcional para compat retroativa  
  restaurant Restaurant? @relation("AuditLogs", fields: [restaurantId], references: [id], onDelete: SetNull)  
  
  @@index([restaurantId]) // NOVO  
  @@index([userId]) // NOVO  
  @@index([action]) // NOVO  
  @@index([entityType, entityId]) // NOVO  
  @@index([createdAt]) // NOVO  
}
```

Restaurant Model

```
model Restaurant {  
  // ... relações existentes  
  auditLogs AuditLog[] @relation("AuditLogs") // NOVA  
}
```

Migração (Backwards Compatible)

```
yarn prisma db push # sem --accept-data-loss  
yarn prisma generate
```

Impactos

- **Isolamento multi-tenant:** AuditLogs agora podem ser filtrados por restaurante
- **Performance:** 5 novos índices otimizam consultas comuns
- **LGPD:** Melhor rastreabilidade de ações por tenant
- **Retrocompat:** restaurantId é opcional, não quebra dados existentes

Fix em Testes

Ajustado `setup-after-env.ts` para limpar RecipeIngredient via relação parent:

```
await prisma.recipeIngredient.deleteMany({
  where: { recipe: { restaurantId: restaurant.id } }
});
```

PHASE 42 - Monitoramento Avançado (Sentry)

Novos Recursos em `lib/sentry.ts`

Contexto Multi-Tenant

```
setRestaurantContext(restaurantId, restaurantName, plan);
// Define tags: restaurant.id, restaurant.name, restaurant.plan
```

Tracking de Performance

```
await trackPerformance('fetchOrders', 'db.query', async () => {
  return await prisma.order.findMany(...);
});
// Auto-alerta se operação > 3 segundos
```

Breadcrumbs Estruturados

- `trackApiCall()` - rastreia chamadas API
- `trackDbQuery()` - rastreia queries DB
- `addBreadcrumb()` - breadcrumbs customizados

Alertas de Negócio

- `trackFailedPayment()` - pagamentos falhados
- `trackStockAlert()` - estoque crítico
- `trackAuthFailure()` - tentativas de login falhadas
- `trackRateLimitHit()` - bloqueios por rate-limit

Error Boundary React

Novo arquivo: `components/error-boundary.tsx`

```
<ErrorBoundary name="DashboardSection">
  <MyComponent />
</ErrorBoundary>

// Ou simples:
<SectionErrorBoundary sectionName="Gráfico de Vendas">
  <SalesChart />
</SectionErrorBoundary>
```

Configs Aprimoradas

`sentry.client.config.ts`

- `browserTracingIntegration()` - performance front-end
- `tracePropagationTargets` - correlação server-client
- `maxBreadcrumbs: 100` (era 50)

- Ignore: ResizeObserver, Hydration errors, extensions

sentry.server.config.ts

- profilesSampleRate: 0.1 (prod)
- Ignore: P2025 (Prisma not-found), ECONNREFUSED, health checks

PHASE 43 - Documentação da API (Swagger/OpenAPI)

Dependências Instaladas

```
yarn add next-swagger-doc swagger-ui-react
yarn add -D @types/swagger-ui-react @types/swagger-jsdoc
```

Arquivos Criados

Arquivo	Descrição
lib/swagger.ts	Configuração OpenAPI 3.0 + schemas
app/api/docs/route.ts	Endpoint /api/docs (JSON spec)
app/docs/page.tsx	Página /docs (Swagger UI)
app/docs/swagger-ui.tsx	Componente client-side wrapper

Schemas Documentados

OpenAPI 3.0 inclui schemas para:

- Ingredient - ingredientes
- Recipe - receitas
- Order + OrderItem - pedidos
- Supplier - fornecedores
- StockMovement - movimentações de estoque
- Error - padrão de erro

Tags Organizacionais

- Auth, Ingredients, Recipes, Stock, Orders
- Suppliers, Reports, Restaurant, Users, Billing

Security Schemes

```
securitySchemes:
  sessionAuth:
    type: apiKey
    in: cookie
    name: next-auth.session-token
  bearerAuth:
    type: http
    scheme: bearer
```

Respostas Padronizadas

- `Unauthorized` (401)
- `Forbidden` (403)
- `NotFound` (404)
- `BadRequest` (400)
- `InternalServerError` (500)

Endpoints Documentados (via JSDoc)

Exemplos de JSDoc adicionados em:

- `app/api/ingredients/route.ts` (GET + POST)
- `app/api/recipes/route.ts` (GET)
- `app/api/docs/route.ts` (especificação)

Acesso

- **JSON Spec:** <https://restaurantes-cl3480.abacusai.app/api/docs>
- **Swagger UI:** <https://restaurantes-cl3480.abacusai.app/docs>

Métricas e Verificações

TypeScript

- ✓ Compilação sem erros (`npx tsc --noEmit`)

Prisma

- ✓ Schema em sync com banco
- ✓ Client regenerado

Build

- ✓ Next.js build compila todas as 94 páginas

Próximos Passos Sugeridos

Monitoramento

1. Configurar DSN do Sentry em produção (`SENTRY_DSN` , `NEXT_PUBLIC_SENTRY_DSN`)
2. Criar alertas customizados no dashboard Sentry para:
 - Payment failures > 5/hora
 - Stock critical alerts > 10/hora
 - Auth failures > 20/hora
3. Aplicar `setRestaurantContext()` no middleware

Documentação

1. Adicionar JSDoc em todos os endpoints restantes (>100 endpoints)
2. Adicionar exemplos de request/response
3. Versionamento da API (v1, v2)
4. Documentação de webhooks

Schema

1. Popular `restaurantId` em registros existentes da AuditLog
 2. Considerar tornar `restaurantId` obrigatório após migração de dados
 3. Adicionar índices similares em outros modelos de alto volume
-

Checklist Final

- [x] Schema AuditLog atualizado com `restaurantId` + 5 indexes
- [x] Restaurant model com relação inversa `auditLogs`
- [x] `yarn prisma db push` executado (compatível)
- [x] `yarn prisma generate` executado
- [x] `lib/sentry.ts` expandido com 15+ helpers
- [x] Componente `ErrorBoundary` criado
- [x] Sentry configs aprimoradas (client + server)
- [x] Swagger/OpenAPI instalado e configurado
- [x] Endpoint `/api/docs` funcional
- [x] Página `/docs` com Swagger UI
- [x] JSDoc em endpoints principais
- [x] TypeScript compila sem erros
- [x] Build Next.js passa